

Aircraft Hangar Positioning: A Mixed-Integer Programming Model and Heuristic Solving Approaches

May 26, 2026

Abstract

We study the *aircraft hangar positioning problem* (AHPP): given a hangar with a set of maintenance positions connected by directed *blocking arcs*, a fleet of aircraft each with a known total maintenance duration, and time-window constraints on start and finish, assign every aircraft to exactly one position and schedule its maintenance block so as to minimise a weighted combination of total delay, makespan, and manoeuvring movements. The blocking-arc structure captures physical constraints: an aircraft in a *front* position obstructs entry to and exit from a *rear* position, incurring costly movements whenever the occupancy windows of the two aircraft overlap. We present a compact mixed-integer linear programming (MILP) formulation, expressed at the aircraft level so that each aircraft is treated as a single uninterrupted maintenance block, together with two complementary heuristic approaches: a topology-aware GRASP that steers heavy aircraft away from high-load positions, and a fixed-assignment scheduler that re-optimises timing given a position assignment. Combining these in a *safe pipeline* always yields a solution at least as good as any individual component. Computational experiments on 120 benchmark instances covering six hangar topologies, three slack levels, and up to 30 aircraft show that the heuristic pipeline consistently outperforms the exact solver within a 60-second budget as soon as the fleet grows beyond the regime in which the exact solver is competitive, while the exact solver closes the gap on the smallest instances.

Keywords: aircraft hangar scheduling; mixed-integer linear programming; GRASP heuristic; blocking arc constraints; maintenance planning.

1 Introduction

Aircraft maintenance hangars represent critical bottlenecks in airline operations. Every hour an aircraft spends waiting for a maintenance slot translates directly into revenue loss and schedule disruption. Yet the spatial organisation of maintenance positions is rarely treated as a first-class optimisation problem: planners typically rely on experience-based rules or simple priority queues, leaving significant performance improvements untapped.

The core difficulty arises from the *blocking structure* of the hangar. In a typical configuration, aircraft parked in *front* positions physically obstruct the entry and exit of aircraft parked in

rear positions behind them. Whenever a rear-position aircraft needs to enter or leave while a front-position aircraft is being serviced, at least one *manoeuvring movement*—towing or pushing the front aircraft out of the way—is required. Such movements are time-consuming, require ground-crew coordination, and carry a risk of damage. Minimising them is therefore a hard operational concern, not merely a secondary objective.

Formally, the *aircraft hangar positioning problem* (AHPP) requires simultaneously deciding (1) which *position* in the hangar to assign each arriving aircraft to and (2) when to *schedule* its maintenance block within that position, subject to earliest-start and target-finish time-windows, minimum separation times between consecutive aircraft at the same position, and blocking-arc constraints that link the schedules of aircraft at spatially related positions. Each aircraft is treated as a single uninterrupted maintenance block of known total duration: the operationally relevant quantities (makespan, delays, blocking interactions) depend only on this aggregated block, and the internal sequence of maintenance steps of each aircraft can be reconstructed deterministically once its start time is fixed.

The AHPP belongs to the broad family of combined assignment and scheduling problems. It combines elements of the *resource-constrained project scheduling problem* (RCPSP) [Blazewicz et al., 1983], classical *machine scheduling with blocking* [Hall and Shmoys, 1996], and *layout-dependent sequence-dependent set-up times* [Pinedo, 2016]. What distinguishes the AHPP from these antecedents is the *directed blocking-arc graph*: the interference between two aircraft is asymmetric (front blocks rear, not vice versa), it depends on the positions assigned, and it generates movements that are themselves part of the objective function rather than merely set-up penalties.

The contributions of this paper are the following. We introduce a precise graph-based model of hangar topology built on *blocking arcs* and a formal definition of the entry and exit movements they induce (§3). On top of this model we present a compact mixed-integer linear programming formulation in which blocking movements are encoded through a six-binary indicator that captures both entry and exit interactions (§4). To overcome the limited scalability of the exact formulation we propose a topology-aware GRASP that uses a *blocking-load* score per position to bias heavy aircraft away from topologically central positions, with multi-start diversification, together with a fixed-assignment scheduler (FAS) that re-optimises timing given a fixed or partially fixed position assignment, and a composition strategy that guarantees the final solution is never inferior to the topology heuristic alone (§5). Finally, we design a systematic benchmark with six hangar topologies—from fully independent to maximally constrained—three slack levels, and four fleet sizes up to 30 aircraft, comprising 120 instances that permit a controlled analysis of algorithm behaviour (§6).

The rest of the paper is organized as follows. Section 3 formally defines the AHPP. Section 4 presents the MILP formulation. Section 5 describes the heuristic solving approaches. Section 6 reports the computational benchmark. Section 7 concludes and outlines future directions.

2 Related Work

The AHPP lies at the intersection of four research streams that we briefly review here: aircraft hangar maintenance scheduling with spatial layout, broader aircraft maintenance scheduling at

the fleet or line level, airport ground operations and stand or gate allocation, and methodological precedents from MILP scheduling with time-window or blocking constraints and from GRASP metaheuristics for combined assignment and scheduling problems. The first stream is the closest in application domain and supplies our main point of comparison; the second positions the AHPP within aircraft maintenance optimisation at large; the third provides adjacent airport assignment and recovery models; the fourth contributes the modelling and algorithmic toolkit. We do not address long-term aircraft maintenance routing or check scheduling, which decide when aircraft enter maintenance over a planning horizon; the AHPP starts from a set of accepted maintenance requests and optimises their spatial-temporal placement inside a single hangar.

The closest body of work is the line of MILP formulations for the integrated hangar maintenance scheduling and layout problem. [Qin et al. \[2017\]](#) introduce the first MILP that couples maintenance scheduling and multi-period parking layout with non-overlapping constraints between aircraft. Building on that line, [Qin et al. \[2018a\]](#) develop a two-stage exact-heuristic procedure for aircraft parking stand allocation that trades off expected revenue against safety distance, and [Qin et al. \[2018b\]](#) extend it with revised No-Fit Polygons and a family of heuristic-based valid inequalities for the safety-margin objective. [Qin et al. \[2019\]](#) broaden the view to joint maintenance scheduling and parking under a hangar with time-varying capacity and blocking along taxi routes, proposing an event-based discrete-time MILP and a rolling-horizon heuristic for large instances, and [Qin et al. \[2020\]](#) integrate scheduling, layout, movement trajectories and multi-skill technician assignment under an MRO outsourcing mode through a two-stage decomposition. More recently, the preprint of [Farhang Pazhooh and Shams Shemirani \[2025\]](#) proposes a continuous-time MILP for integrated hangar scheduling and layout, reporting substantial computational improvements over previous event-based formulations and experiments on larger instances. These works share with the AHPP the assign-and-schedule decision structure but model the spatial interactions through geometric primitives — No-Fit Polygons for non-overlapping placement, taxi-route capacities, safety distances, or overlap of continuous-time windows — so that the operational impact of moving an aircraft is captured indirectly via route feasibility, parking-plan feasibility, or movement-path constraints, rather than as an explicit cost term induced by an asymmetric, directed relation between positions.

Stepping out of the hangar but staying in aircraft maintenance, [Shaukat et al. \[2020\]](#) formulate the line maintenance scheduling problem as a MIP that places maintenance work in the layovers between scheduled flights through integrated exact and sequential heuristic approaches; [Witteman et al. \[2021\]](#) cast multi-year task allocation across an airline’s fleet as a time-constrained variable-sized bin-packing problem and solve 45-aircraft instances with a worst-fit-decreasing heuristic; and [Torres Sanchez et al. \[2020\]](#) provide a multi-objective MILP that integrates airline fleet maintenance scheduling with tail assignment and workshop capacity over 30-day horizons with 529 aircraft and eight workshops. These works tackle rich temporal and resource structures at the fleet or network scale but abstract away the within-hangar geometry that drives the AHPP: positions are pooled into workshop capacities or maintenance slots, not laid out as a graph of directed blocking relationships. The AHPP is therefore complementary, operating at the operational time scale of a single hangar.

At the broader scale of airport ground operations, the systematic review of [Dahanayaka](#)

et al. [2026] surveys 199 articles on gate and stand allocation, turnaround scheduling, workforce assignment, and surface movement. Among the relevant models, Guépet et al. [2015] solve the airport stand allocation problem with both an exact MILP and spatial and time decomposition heuristics; their setting is the closest methodological cousin to the AHPP — aircraft-to-position assignment plus timing, with shadow restrictions and tow movements as soft objective terms — and their two major European airport datasets illustrate the same exact-versus-decomposition trade-off that drives our solver portfolio. Asadi et al. [2021] formulate the gate-assignment schedule-recovery problem with constrained resources; Evler et al. [2021a,b] extend the approach to recover airline ground operations under uncertain arrival times; and Yao et al. [2024] couple stand allocation with ground-support-vehicle scheduling. All these works share with the AHPP the aircraft-to-position assignment layer and the soft-time-window machinery, but operate at the apron level with shared servicing resources rather than on a hangar with directed blocking between maintenance positions.

The AHPP also draws on a broader literature on MILP scheduling with time windows and blocking-like interactions outside the airport domain. Mao et al. [2018] address multi-degree and multi-hoist cyclic scheduling with time windows, where hoists share a track and must avoid spatial collisions; their disjunctive-time-window approach is structurally similar to our blocking-arc constraints. Lee et al. [2024] model multi-satellite mission and communication scheduling as a MILP with time-window and conflict constraints, and Bayá et al. [2022] apply MILP to a grain facility with shared storage capacity. None of these works expresses interference as a directed asymmetric relation tied to a position-assignment decision, but they illustrate the modelling techniques — time windows, conflict graphs, big- M disjunctions — that the AHPP combines. On the algorithmic side, the topology-aware heuristic developed in this paper is a Greedy Randomized Adaptive Search Procedure; its foundations go back to Feo and Resende [1995] and are surveyed in the annotated bibliographies of Festa and Resende [2002, 2009]. Closer to our setting, Wang et al. [2024] attack a yard-crane scheduling problem with dynamic assignment of input/output points using construction and local-search operators that mirror our coupling between position selection and start-time choice, and Ferone et al. [2018] provide the methodological basis for the biased-randomisation step in our α -controlled restricted candidate list.

Against this background, the present paper contributes along two fronts. On the *modelling* side, it abstracts the hangar layout as a directed blocking-arc graph on maintenance positions, making the asymmetry between front and rear — and its dependence on the assignment decision — an explicit feature of the problem rather than an indirect consequence of taxi routes, safety distances, or geometric overlap as captured by No-Fit Polygons [Qin et al., 2018a,b, 2019, 2020, Farhang Pazhooh and Shams Shemirani, 2025]. The entry and exit manoeuvres induced by this graph then become an optimisable objective component through a compact six-binary indicator structure, so that movements are minimised jointly with delay and makespan through user-defined weights, rather than appearing only as feasibility side-effects of capacities or distances.

On the *algorithmic* side, the paper combines an aircraft-level MILP — where each aircraft is a single uninterrupted maintenance block — with a topology-aware GRASP and a fixed-assignment scheduler glued together by a safe pipeline that returns the best solution found by its components. The exact solver is competitive on small instances, in line with the scale at

which the formulations of Guépet et al. [2015], Shaukat et al. [2020], Farhang Pazhooh and Shams Shemirani [2025] also close to optimality, while the safe pipeline takes over as the fleet size grows: the crossover between exact and heuristic methods in our benchmark sits near $R = 14$ aircraft for a 60-second wall-clock budget, consistent with the 14–25-aircraft threshold reported across the hangar literature for exact MILP optimality [Farhang Pazhooh and Shams Shemirani, 2025, Guépet et al., 2015, Shaukat et al., 2020, Qin et al., 2019]. Rather than competing directly with general continuous-time layout formulations on their own benchmark classes, the focus is on a topology-driven blocking abstraction and on a fast solver portfolio that delivers high-quality feasible solutions on a controlled benchmark of congested hangar layouts, in the regime where the exact solver leaves large optimality gaps.

3 Problem Statement

We model a hangar as a directed graph $G = (\mathcal{P}, \mathcal{A})$, where $\mathcal{P} = \{p_1, \dots, p_P\}$ is the finite set of *maintenance positions* and $\mathcal{A} \subseteq \mathcal{P} \times \mathcal{P}$ is a set of *blocking arcs*. An arc $(p, p') \in \mathcal{A}$ means that position p (*front*) physically blocks access to position p' (*rear*): an aircraft at position p' cannot enter or leave the hangar without temporarily displacing the aircraft at position p . In practice G is acyclic, so a position cannot block itself transitively, and we define the *blocking load* of a position as $\ell(p) = |\{q \in \mathcal{P} : p \text{ can reach } q \text{ in } G\}|$. A higher blocking load means that aircraft at p can obstruct more other positions, making p topologically *central*.

Definition 1 (Blocking arc). *An arc $(p, p') \in \mathcal{A}$ induces an entry movement whenever an aircraft arrives at position p' while another aircraft is being serviced at position p , and an exit movement whenever an aircraft departs from position p' while another aircraft is still at position p . Each movement contributes 2 to the movement count, one for the displacement of the front aircraft and one for its return.*

Let $\mathcal{R} = \{r_1, \dots, r_R\}$ be the set of aircraft. Each aircraft $r \in \mathcal{R}$ requires a fixed amount of maintenance work of total duration $D_r > 0$, which we treat as atomic: while r occupies a position, the position is busy for exactly D_r time units in a single uninterrupted block.¹ Each aircraft has an *earliest start* $E_r \geq 0$ before which work cannot begin and a *target finish* $L_r > E_r$ beyond which completion incurs delay.

A solution to the AHPP makes two coupled decisions. At the assignment level, it sends each aircraft r to exactly one position $\pi(r) \in \mathcal{P}$. At the scheduling level, it fixes a start time $s_r \geq 0$ for every aircraft, from which the finish time follows as $f_r = s_r + D_r$.

Any solution must satisfy the following requirements. Every aircraft is *assigned to exactly one position*, and a position handles its aircraft *sequentially*: when two aircraft r and r' are assigned to the same position, their occupancy intervals $[s_r, f_r]$ and $[s_{r'}, f_{r'}]$ must not overlap and a minimum separation $\varepsilon > 0$ must elapse between the finish of one and the start of the next. Each aircraft’s start time must respect its *earliest-start time window*, $s_r \geq E_r$.

¹In practice the maintenance work consists of an ordered sequence of jobs, and the per-job timings can be reconstructed deterministically once the aircraft start time is known (see §4). Because the operationally relevant quantities (makespan, delay, blocking interactions) depend only on the aggregated aircraft block, the problem can be stated entirely at the aircraft level.

The blocking-arc structure imposes the final family of requirements. For every arc $(p, p') \in \mathcal{A}$ and every pair of aircraft r, r' with $\pi(r) = p$ and $\pi(r') = p'$, an entry movement is incurred if r' arrives at the rear position while r is still being serviced at the front, that is, if $s_r \leq s_{r'} \leq f_r - \varepsilon$; analogously, an exit movement is incurred whenever r' departs from the rear while r is still at the front, that is, if $s_r \leq f_{r'} \leq f_r - \varepsilon$. Each such event triggers a temporary relocation of the front aircraft and is counted as two movements (displacement plus return).

Single ε for two roles. The parameter ε plays the same physical role in both of the requirements above: it is the time needed to tow an aircraft in or out of a hangar slot. This primitive operation has the same duration whether it is performed as a turnaround between two consecutive maintenance slots at one position — the role of ε in the separation requirement — or as a temporary displacement of the front aircraft to free a blocked rear position — the role of ε in the $f_r - \varepsilon$ margin of the blocking conditions. We therefore treat the two appearances as a single parameter. Throughout this work all temporal magnitudes are expressed in days; for the reported benchmark we use $\varepsilon = 0.5$ days, an indicative half-day allowance for this primitive towing operation. The value is recorded in each instance file, so that experiments on hangars with different operational rhythms can use a different ε without changing the model.

The objective is a weighted combination of three criteria,

$$\min W^M \cdot m + W^D \cdot \sum_{r \in \mathcal{R}} v_r^D + W^S \cdot n, \quad (1)$$

where $m = \max_{r \in \mathcal{R}} f_r$ is the makespan, $v_r^D = \max(0, f_r - L_r)$ is the delay of aircraft r , n counts the blocking movements induced above, and $W^M, W^D, W^S \geq 0$ are user-defined weights.

Remark 1. *Throughout the experiments we adopt as default $(W^M, W^D, W^S) = (0.1, 1.0, 10)$, which encodes the operational priority order movements $\succ$ delay $\succ$ makespan. Section 6 also evaluates a delay-priority profile $(1, 10, 0.1)$ and a makespan-priority profile $(10, 0.1, 1)$ to expose the sensitivity of the solutions to this choice.*

The AHPP is $\mathcal{NP}$ -hard, since it subsumes the single-machine weighted-completion-time scheduling problem with arbitrary release dates, which is already $\mathcal{NP}$ -hard [Lenstra et al., 1977]; the additional position-assignment layer and the blocking-arc constraints further enlarge the search space. For a benchmark instance with $P = 5$ positions and $R = 10$ aircraft, the MILP formulation of §4 involves several thousand binary variables and an order of magnitude more constraints; at $R = 30$ these grow by an additional order of magnitude, motivating the heuristic approaches of §5.

4 MILP Formulation

We now present a mixed-integer linear programming formulation of the AHPP aligned with the aircraft-level view of §3: each aircraft r is treated as a single atomic block $[s_r, s_r + D_r]$ with total processing time D_r taken from the instance data, and all blocking interactions are expressed in terms of the aircraft start and finish times s_r, f_r . Once an optimal (s_r^*, x_{rp}^*) is obtained, the underlying per-job timings of each aircraft can be reconstructed deterministically by walking the

precedence chain of its maintenance work, as described in §4.1. The model is implemented in Pyomo [Bynum et al., 2021] and solved with Gurobi [Gurobi Optimization, LLC, 2023].

Throughout this section parameters are written with upper-case Latin letters and decision variables with lower-case Latin letters, with Greek letters reserved for fixed scalar constants such as the minimum separation ε and the big- M relaxation M .

Sets

$\mathcal{P}$	maintenance positions
$\mathcal{R}$	aircraft
$\mathcal{A} \subseteq \mathcal{P} \times \mathcal{P}$	blocking arcs (p, p') : position p blocks position p'

Parameters

D_r	total processing time of aircraft r (sum of its job durations)
E_r	earliest start time of aircraft r
L_r	target finish time (due date) of aircraft r ; soft, completion beyond L_r is penalised but not forbidden
ε	tow-in / tow-out time, used both as separation between consecutive aircraft at the same position and as the safety margin $f_r - \varepsilon$ in the blocking conditions; per-instance value read from the input
H^{UB}	technical upper bound on every aircraft start and finish time, $H^{UB} = \max_{r \in \mathcal{R}} E_r + \sum_{r \in \mathcal{R}} D_r + (\mathcal{R} - 1)\varepsilon$
M	big- M constant; we take $M = H^{UB} + \varepsilon$
W^M, W^D, W^S	weights for makespan, total delay, and movements

The bound H^{UB} is valid because the schedule that processes every aircraft sequentially on a single position, starting at $\max_r E_r$ and inserting a separation of ε between consecutive aircraft, is feasible and has makespan exactly H^{UB} . This provides a valid modelling horizon for the MILP without imposing an operational horizon on the benchmark. With $s_r, f_r \in [0, H^{UB}]$, taking $M = H^{UB} + \varepsilon$ is sufficient to relax the sequencing constraints when their guard binaries are inactive: the residual right-hand side in (5) (resp. (11), (17)) is at most $f_r + \varepsilon - M \leq 0$, so the corresponding constraint becomes vacuous. The same value also relaxes the blocking indicator constraints whenever any of their position or comparison guards are not active.

Variables

Assignment and timing. Continuous variables $s_r, f_r \in [0, H^{UB}]$ are the start and finish times of aircraft r at its assigned position. The binary variables are x_{rp} (1 if aircraft r is assigned to position p) and $q_{rr'p}$ (1 if aircraft r precedes aircraft r' at position p).

Blocking. For every pair of distinct aircraft $r \neq r'$ and every blocking arc $(p, p') \in \mathcal{A}$, we introduce six binary variables. On the entry side, $\alpha_{rr'pp'}^{\text{in}} = 1$ iff $s_{r'} > s_r$, $\beta_{rr'pp'}^{\text{in}} = 1$ iff $s_{r'} \leq f_r - \varepsilon$, and $u_{rr'pp'}^{\text{in}} = 1$ iff the entry of r' at p' forces displacing r at p . The exit-side variables $\alpha_{rr'pp'}^{\text{out}}, \beta_{rr'pp'}^{\text{out}}, u_{rr'pp'}^{\text{out}}$ are defined symmetrically by replacing $s_{r'}$ with $f_{r'}$ in the conditions above.

Objective. The makespan $m \in \mathbb{R}_{\geq 0}$, the per-aircraft delay $v_r^D \in \mathbb{R}_{\geq 0}$, and the total number of movements $n \in \mathbb{Z}_{\geq 0}$ collect the three components of the objective.

Constraints

To keep the constraints compact, we abbreviate the universal quantifiers $\forall r \neq r' \in \mathcal{R}, (p, p') \in \mathcal{A}$ by $(\dagger)$ and $\forall r \neq r' \in \mathcal{R}, p \in \mathcal{P}$ (same-position pair) by $(\ddagger)$. The full model is:

$$\sum_{p \in \mathcal{P}} x_{rp} = 1 \quad \forall r \in \mathcal{R} \quad (2)$$

$$f_r = s_r + D_r \quad \forall r \in \mathcal{R} \quad (3)$$

$$s_r \geq E_r \quad \forall r \in \mathcal{R} \quad (4)$$

$$\begin{aligned} s_{r'} &\geq f_r + \varepsilon - M(1 - q_{rr'p}) \\ &\quad - M(1 - x_{rp}) - M(1 - x_{r'p}) \end{aligned} \quad (\ddagger) \quad (5)$$

$$q_{rr'p} + q_{r'rp} \geq x_{rp} + x_{r'p} - 1 \quad \forall r < r' \in \mathcal{R}, p \in \mathcal{P} \quad (6)$$

$$q_{rr'p} + q_{r'rp} \leq 1 \quad \forall r < r' \in \mathcal{R}, p \in \mathcal{P} \quad (7)$$

$$s_{r'} \geq s_r - M(1 - \alpha_{rr'pp'}^{\text{in}}) \quad (\dagger) \quad (8)$$

$$s_{r'} \leq s_r + M \alpha_{rr'pp'}^{\text{in}} \quad (\dagger) \quad (9)$$

$$s_{r'} \leq f_r - \varepsilon + M(1 - \beta_{rr'pp'}^{\text{in}}) \quad (\dagger) \quad (10)$$

$$\begin{aligned} s_{r'} &\geq f_r - M \beta_{rr'pp'}^{\text{in}} \\ &\quad - M(1 - x_{rp}) - M(1 - x_{r'p'}) \end{aligned} \quad (\dagger) \quad (11)$$

$$u_{rr'pp'}^{\text{in}} \leq \min\{x_{rp}, x_{r'p'}, \alpha_{rr'pp'}^{\text{in}}, \beta_{rr'pp'}^{\text{in}}\} \quad (\dagger) \quad (12)$$

$$\begin{aligned} u_{rr'pp'}^{\text{in}} &\geq x_{rp} + x_{r'p'} \\ &\quad + \alpha_{rr'pp'}^{\text{in}} + \beta_{rr'pp'}^{\text{in}} - 3 \end{aligned} \quad (\dagger) \quad (13)$$

$$f_{r'} \geq s_r - M(1 - \alpha_{rr'pp'}^{\text{out}}) \quad (\dagger) \quad (14)$$

$$f_{r'} \leq s_r + M \alpha_{rr'pp'}^{\text{out}} \quad (\dagger) \quad (15)$$

$$f_{r'} \leq f_r - \varepsilon + M(1 - \beta_{rr'pp'}^{\text{out}}) \quad (\dagger) \quad (16)$$

$$\begin{aligned} f_{r'} &\geq f_r - M \beta_{rr'pp'}^{\text{out}} \\ &\quad - M(1 - x_{rp}) - M(1 - x_{r'p'}) \end{aligned} \quad (\dagger) \quad (17)$$

$$u_{rr'pp'}^{\text{out}} \leq \min\{x_{rp}, x_{r'p'}, \alpha_{rr'pp'}^{\text{out}}, \beta_{rr'pp'}^{\text{out}}\} \quad (\dagger) \quad (18)$$

$$\begin{aligned} u_{rr'pp'}^{\text{out}} &\geq x_{rp} + x_{r'p'} \\ &\quad + \alpha_{rr'pp'}^{\text{out}} + \beta_{rr'pp'}^{\text{out}} - 3 \end{aligned} \quad (\dagger) \quad (19)$$

$$n = 2 \sum_{(p,p') \in \mathcal{A}} \sum_{\substack{r,r' \in \mathcal{R} \\ r \neq r'}} (u_{rr'pp'}^{\text{in}} + u_{rr'pp'}^{\text{out}}) \quad (20)$$

$$v_r^D \geq f_r - L_r \quad \forall r \in \mathcal{R} \quad (21)$$

$$m \geq f_r \quad \forall r \in \mathcal{R} \quad (22)$$

Constraints (2)–(4) pin each aircraft to a single position, link its finish time to the duration parameter D_r , and enforce the earliest-start window. The same-position block (5)–(7) uses the

ordering binary $q_{rr'p}$ to select the disjunction direction whenever two aircraft share a position: (5) forces $s_{r'} \geq f_r + \varepsilon$ when $q_{rr'p} = 1$ and both r, r' are at p (the position guards relax the constraint otherwise), while (6)–(7) guarantee that exactly one direction is active when the position is shared and that both directions cannot hold simultaneously. The entry-blocking block defines two indicators and combines them: (8)–(9) make $\alpha_{rr'pp'}^{\text{in}} = 1$ iff $s_{r'} > s_r$, while (10)–(11) make $\beta_{rr'pp'}^{\text{in}} = 1$ iff $s_{r'}$ falls inside the servicing window of r , with the position guards $-M(1 - x_{rp}) - M(1 - x_{r'p'})$ in (11) switching the constraint off whenever the blocking placement does not actually hold; (12)–(13) then set $u_{rr'pp'}^{\text{in}}$ to the conjunction $x_{rp} \wedge x_{r'p'} \wedge \alpha_{rr'pp'}^{\text{in}} \wedge \beta_{rr'pp'}^{\text{in}}$, so an entry movement is counted iff both aircraft are at the relevant blocking positions and r' enters during the servicing window of r . The exit-blocking block (14)–(19) mirrors the entry block with $f_{r'}$ in place of $s_{r'}$. Finally, (20) aggregates all entry and exit indicators into the movement count n (the factor 2 accounts for displacement plus return), (21) lower-bounds the positive delay of each aircraft by $f_r - L_r$, and (22) lower-bounds the makespan by every aircraft finish time.

Objective

$$\min \quad W^M m + W^D \sum_{r \in \mathcal{R}} v_r^D + W^S n. \quad (23)$$

4.1 Job-level recovery

The atomic-block view of §3 is sufficient for the optimisation model itself. In deployment, however, the maintenance work of each aircraft typically consists of an ordered sequence of jobs $j_1^T, j_2^T, \dots$ with individual durations summing to D_r . Given an aircraft-level solution (s_r^*, x_{rp}^*) , those per-job start times are recovered deterministically by walking the chain: the first job of r starts at s_r^* , and each subsequent job starts at the finish of its predecessor. Since none of the constraints above involve individual job times, this recovery is loss-free: every feasible aircraft-level solution yields a unique per-job schedule with the same makespan, total delay, and movement count.

5 Solving Approaches

The MILP of §4 is exact but its size grows quickly with the number of aircraft and blocking arcs. We therefore develop three complementary methods—a topology-aware GRASP, a fixed-assignment scheduler, and a safe pipeline composition—that operate at different abstraction levels: the GRASP works on the position assignment, the fixed-assignment scheduler refines the timing given an assignment, and the safe pipeline combines them.

5.1 Topology-Aware GRASP

The driving observation behind this method is that assigning a heavy aircraft (large D_r) to a position with high outgoing blocking degree forces its long occupancy window to overlap with aircraft serviced behind it, triggering movements. We formalise this through the *blocking load* of a position:

$$\ell(p) = |\{q \in \mathcal{P} : p \text{ reaches } q \text{ in } G\}|,$$

i.e. the number of positions that lie downstream of p in the blocking-arc graph. The construction step penalises each candidate pair (r, p) proportionally to $D_r \cdot \ell(p)$, steering heavy aircraft towards positions with low (ideally zero) blocking load.

Given the cost function

$$c(r, p) = W^D \hat{v}_r^D + W^M \hat{m} + W^S \hat{n}_{rp} + W^T D_r \ell(p),$$

in which hatted quantities are estimated by a fast greedy scheduler and W^T is the topology penalty weight, the heuristic constructs a feasible solution by biased-random greedy insertion, then improves it through a local-search portfolio. Diversification is obtained by restarting from independent random seeds. Algorithm 1 states the full procedure.

Algorithm 1: Topology-aware GRASP with multi-start (n_s restarts).

Input: Instance I , weights (W^M, W^D, W^S, W^T) , RCL parameter α , time budget T , number of starts n_s .
Output: Best solution z^* found.

```

1  $z^* \leftarrow +\infty$ ;  $T_s \leftarrow T/n_s$ 
2 for  $k = 1, \dots, n_s$  do
3    $\mathcal{U} \leftarrow \mathcal{R}$  sorted by  $D_r$  descending
4   while  $\mathcal{U} \neq \emptyset$  do
5     evaluate  $c(r, p)$  for all  $(r, p) \in \mathcal{U} \times \mathcal{P}_{\text{free}}$ 
6      $\text{RCL} \leftarrow \{(r, p) : c(r, p) \leq c_{\min} + \alpha(c_{\max} - c_{\min})\}$ 
7     pick  $(r, p)$  uniformly from RCL and commit it; remove  $r$  from  $\mathcal{U}$ 
8   end
9    $z \leftarrow$  schedule with current assignment
10  repeat
11    | apply best of: single-move, 2-opt swap, intra-position reorder, perturb-and-rebuild
12  until no operator improves  $z$  or slice  $T_s$  exhausted
13  if  $z < z^*$  then
14    |  $z^* \leftarrow z$ 
15  end
16 end
17 return  $z^*$ 

```

The single-move and 2-opt swap operators reassign one or two aircraft to alternative positions; the intra-position reorder swaps the service sequence of two aircraft sharing a position; the perturb-and-rebuild kick forces a random reassignment of a small subset before re-running the schedule with the same position map. All operators are $O(R^2)$ or cheaper, so the local-search phase converges within the slice budget.

5.2 Fixed-Assignment Scheduler (FAS)

Given a fixed position assignment $\pi : \mathcal{R} \rightarrow \mathcal{P}$, the AHPP reduces to a pure scheduling problem: find aircraft start times $\{s_r\}$ that minimise the objective (23) subject to the constraints (3)–(19), with every x_{rp} replaced by the constant $\mathbf{1}[\pi(r) = p]$. Two simplifications follow. The $O(RP)$ assignment binaries disappear, and the position-guard factors in the blocking constraints collapse to known scalars; the blocking-pair enumeration therefore shrinks from $O(R^2|\mathcal{A}|P^2)$ to $O(R^2|\mathcal{A}|)$.

Job-level timings are recovered from s_r by walking the precedence chain, exactly as in §4.1. The resulting model is encoded directly in `gurobipy`, bypassing the Pyomo layer.

Algorithm 2 summarises the procedure. A multi-candidate extension allows a small set of positions $\kappa(r) \supseteq \{\pi(r)\}$ for each aircraft, recovering a fraction of the full assignment problem while keeping the model tractable. In that variant the candidate set is non-trivial only for the $K = 4$ aircraft with the highest score $\sigma(r) = v_r^D + 0.5 \cdot \mathbf{1}[f_r \approx m]$, and is built as the union of their primary position and the positions they occupy in $K = 10$ diverse topology assignments produced by 1-second GRASP runs.

Algorithm 2: Fixed-Assignment Scheduler (FAS) with optional multi-candidate.

Input: Instance I , assignment π (optionally candidate sets κ), time budget T .

Output: Best feasible schedule found within T .

```

1 if candidates  $\kappa$  provided then
2   |  $\mathcal{Y} \leftarrow \{x_{rp} : p \in \kappa(r)\}$ 
3 end
4 else
5   |  $\mathcal{Y} \leftarrow \emptyset$  (all  $x_{rp}$  fixed to  $\pi$ )
6 end
7 build reduced model with aggregated blocks  $[s_r, s_r + D_r]$  and variables  $\mathcal{Y}$ 
8 solve with Gurobi under time limit  $T$ 
9 recover job start times  $t_j^S$  from  $s_r$  along each precedence chain
10 return resulting schedule

```

5.3 Safe Pipeline

The two preceding methods can be run in sequence: the GRASP produces a position assignment, and FAS refines its timing. Their relative quality is instance-dependent, since neither dominates the other for every configuration. The *safe pipeline* therefore returns the best objective achieved by any of the component methods:

$$z^* = \min_{h \in \mathcal{H}} z_h, \quad \mathcal{H} = \{\text{topology_ms6}, \text{fas_on_topo}\},$$

optionally extended with `fas_2cand`. Since each component already stores its solution in a shared cache, the pipeline performs no additional optimisation; its cost is a single comparison.

Proposition 1. *The safe pipeline objective satisfies $z^* \leq z_h$ for every $h \in \mathcal{H}$; in particular $z^* \leq z_{\text{topology_ms6}}$, so the pipeline never degrades the GRASP result.*

6 Computational Experiments

This section describes the benchmark design and instance generation procedure. Detailed numerical results will be presented in subsequent sections once the full experimental run is complete.

6.1 Benchmark Design

The benchmark is designed to isolate the effect of four axes of variation:

1. **Topology** (six levels): the structure of the blocking-arc graph $G = (\mathcal{P}, \mathcal{A})$.
2. **Fleet size** (four levels): $R \in \{5, 10, 20, 30\}$ aircraft.
3. **Slack** (three levels): how much temporal headroom aircraft have between their processing time and their time window.
4. **Maximum constraint density**: a hard large-scale configuration that combines maximum blocking with a large fleet.

All instances use $P = 5$ hangar positions. The benchmark comprises 12 configurations, each replicated with 10 independently seeded random instances, yielding **120 instances** in total. Table 1 shows the full specification.

Table 1: Benchmark specification: 12 configurations $\times$ 10 seeds = 120 instances. All instances use $P = 5$ positions.

Axis	Topology	R	Slack	Seeds	Purpose
Topology	None	10	tight	1–10	Baseline (no blocking)
	Chain	10	tight	1–10	Linear blocking
	Hub	10	tight	1–10	Star topology
	Triangle	10	tight	1–10	Classic hangar layout
	Two rows	10	tight	1–10	Parallel rows
	Full	10	tight	1–10	Maximum blocking
Size	Triangle	5	tight	1–10	Small (MILP-tractable)
	Triangle	20	tight	1–10	Medium (heuristic regime)
	Triangle	30	tight	1–10	Large
Slack	Triangle	10	loose	1–10	Relaxed time windows
	Triangle	10	medium	1–10	Moderate time windows
Hard large	Full	20	tight	1–10	Dense + large

6.2 Instance Generation

For each configuration of Table 1, the 10 random seeds drive a deterministic generator that produces the hangar topology, the fleet of aircraft, and their associated jobs and time windows. We describe each step below.

6.2.1 Hangar topology families

Each instance is built on top of one of six canonical blocking-arc graphs that together span the full spectrum from fully independent to maximally constrained hangars; this is the source of the topology axis in Table 1. In all cases positions are labelled $\{P1, \dots, P5\}$ in arrival order. The *None* topology takes $\mathcal{A} = \emptyset$, leaving all positions independent and serving as a baseline with no blocking interactions. The *Chain* topology arranges positions in a linear row, $\mathcal{A} = \{(p_i, p_{i+1}) : 1 \leq i < P\}$,

so that each position blocks the next; this represents a single-file hangar. The *Hub* topology designates an *aisle* position p_1 that blocks all others, $\mathcal{A} = \{(p_1, p_k) : k \neq 1\}$, modelling a star layout in which one slot guards the access to the rest. The *Triangle* topology turns the last three positions into a transitive triangle, $\mathcal{A} = \{(a, b), (a, c), (b, c)\}$ with $\{a, b, c\}$ the rear trio, reproducing the layout of the facility that originally motivated this study. The *Two rows* topology splits positions into a front and a rear half and pairs them position-by-position, modelling parallel rows in which only the front aircraft obstructs its corresponding rear one. Finally, the *Full* topology takes every ordered pair as a blocking arc, $\mathcal{A} = \{(p_i, p_j) : i < j\}$, providing the maximally constrained extreme.

These six families were chosen so that each one isolates a distinct qualitative feature of the blocking graph—no blocking, sequential blocking, star blocking, transitive blocking, paired blocking, and dense blocking—while keeping the number of positions fixed at $P = 5$. The resulting topology axis is therefore a controlled probe of how the blocking structure, independently of fleet size and time-window tightness, influences the difficulty of the AHPP and the relative performance of the solving methods.

6.2.2 Aircraft and job generation

Processing times, earliest starts and due dates are sampled on an integer-day grid; the only fractional quantity in the model is the tow-in/tow-out time ε (§3), which is recorded in each instance file and fixed at $\varepsilon = 0.5$ days throughout the reported benchmark. No *operational* horizon is imposed: only makespan and delay enter the objective, so any feasible schedule can be expressed in finite time without an operational upper bound. The MILP formulation does use a technical upper bound H^{UB} (defined in §4) solely to bound the time variables and calibrate the big- M constants, but H^{UB} is derived from the instance data rather than fixed at instance generation, so it is a property of the solver model and not of the benchmark itself.

Given a configuration with R aircraft and a task-count range $[n_{\min}, n_{\max}]$, each aircraft r is generated as follows. Its number of jobs is drawn as $n_r \sim \text{Uniform}\{n_{\min}, \dots, n_{\max}\}$ and the processing time of each job is sampled independently from the discrete uniform $D_j \sim \text{Uniform}\{3, 4, 5, 6, 7, 8\}$ days (mean 5.5, standard deviation ≈ 1.7). The total processing time of the aircraft is the integer sum $D_r = \sum_{j \in \mathcal{J}_r} D_j$ days. The earliest start is sampled uniformly as $E_r \sim \text{Uniform}\{0, \dots, S\}$ days with arrival spread $S = \lceil R\bar{p}/P \rceil$ and $\bar{p} = 5.5$; this scales naturally with the fleet size and the number of positions without introducing an operational horizon. Given the slack ratio ρ of Table 2, the target finish is set to $L_r = E_r + D_r + \max(1, \lceil \rho D_r \rceil)$ days, which guarantees a strictly positive integer-day margin over the processing block. Each aircraft is labelled with one of $n_c = \max(2, \min(6, \lfloor R/2 \rfloor))$ client identifiers, which are kept in the schema to support priority extensions but do not enter the present objective. The jobs of each aircraft are linked into a strict linear precedence chain $j_1^r \rightarrow j_2^r \rightarrow \dots \rightarrow j_{n_r}^r$, and all random draws are controlled by the seed so each instance is fully reproducible from its configuration and seed value.

6.3 Experimental Setup

All MILP and FAS components were modelled with Pyomo [Bynum et al., 2021] and solved with Gurobi 11 [Gurobi Optimization, LLC, 2023]; the topology-aware GRASP was implemented in

Table 2: Slack level definitions. ρ is the ratio of post-window slack to total aircraft duration D_r , applied as $L_r = E_r + D_r + \max(1, \lceil \rho D_r \rceil)$.

Level	ρ
Loose	0.80
Medium	0.35
Tight	0.05

plain Python. Experiments were run on a workstation with an Intel Core i9 processor (16 cores, 32 threads, 3.5 GHz base clock), 32 GB of RAM, and Windows 11 as the operating system. Each run was given a wall-clock time limit of **60 seconds**. Table 3 lists the four methods evaluated, derived from the algorithms of §5. Each method is run under three weight profiles: default $(W^M, W^D, W^S) = (0.1, 1.0, 10)$ (movement-priority), delay-priority $(1, 10, 0.1)$, and makespan-priority $(10, 0.1, 1)$. This yields $4 \times 3 = 12$ solver configurations, applied to all 120 instances, for a total of **1 440 runs**.

Table 3: Solving methods evaluated in the benchmark.

Label	Method	Solver	Notes
<code>milp_baseline</code>	Full MILP	Gurobi (via Pyomo)	Exact; 60 s limit
<code>topology_ms6</code>	GRASP, $n_s=6$	Custom	10 s per start
<code>fas_on_topo</code>	FAS on topology assignment	Gurobi (direct)	60 s; fixed π
<code>safe_pipeline</code>	Best of {topology, FAS}	—	Zero extra cost

6.4 Results

We report mean values over the six random seeds available for each configuration at the time of writing (the full ten-seed run is in progress). The analysis is organised around four questions: (i) up to what size is the MILP provably optimal within the time budget, (ii) how the four methods compare on the resulting feasible solutions, (iii) how the gap between the heuristic pipeline and the exact solver evolves across instance types, and (iv) how the chosen weight profile shapes the trade-off between makespan, delay, and movements.

6.4.1 Reach of the exact MILP

Table 4 reports the MILP outcome under the 60 s budget. The exact solver only closes the $R=5$ configuration to optimality (in about 4 s on average). On every $R=10$ configuration the search exhausts the budget with an average Gurobi gap above 95 % — the gap remains high even on the unconstrained *None* topology (96.2 %), indicating that bound weakness, rather than blocking interactions, is what stalls branch-and-bound. On the medium-size configurations ($R=20$, both triangle and full topologies) the solver overruns the 60 s wall clock by a factor of two to three (an artefact of Gurobi’s post-timeout cleanup) and still ends with a gap close to 100 %; moreover, on five of the six seeds Gurobi aborts before returning any feasible incumbent (for both Triangle and Full $R=20$), so each $R=20$ MILP row aggregates a single usable run. At $R=30$ the model either aborts or exceeds the one-minute Pyomo build cap on every seed, so no MILP row can be

reported for that size; the heuristic pipeline is therefore the only available source of solutions for the largest configuration.

Table 4: MILP performance under the 60s budget per configuration. n = number of usable runs (out of 6 seeds; rows aborted by Gurobi or whose Pyomo build exceeded 60s are excluded); Opt = number of instances proved optimal; $\overline{gap}$ = mean Gurobi optimality gap; $\overline{t_{MILP}}$ = mean wall-clock time (s).

Configuration	n	Opt	$\overline{gap}$ (%)	$\overline{t_{MILP}}$
None ($R=10$, tight)	6	0	96.11	69.1
Chain ($R=10$, tight)	6	0	98.06	69.4
Hub ($R=10$, tight)	6	0	97.73	80.9
Triangle ($R=10$, tight)	6	0	97.05	68.3
Two rows ($R=10$, tight)	6	0	97.08	81.9
Full ($R=10$, tight)	6	0	98.48	68.6
Triangle ($R=5$, tight)	6	6	0.00	4.9
Triangle ($R=20$, tight)	1	0	99.79	194.4
Triangle ($R=30$, tight)	0	—	—	—
Triangle ($R=10$, medium)	6	0	96.45	70.3
Triangle ($R=10$, loose)	6	0	68.17	68.5
Full ($R=20$, tight)	1	0	99.89	156.2

6.4.2 Per-method means by configuration

Table 5 aggregates the four methods under the default profile $(W^M, W^D, W^S) = (0.1, 1, 10)$. Three observations stand out. First, on every $R=10$ configuration with non-trivial blocking the heuristic pipeline finishes well below the best feasible MILP solution found within the budget: **Safe** reduces $\bar{f}$ by 20–40 % over **MILP** on Chain, Hub, Triangle, Two rows and Full, while on the unconstrained *None* topology the two methods are practically tied (107.2 versus 109.3). Within a 60s budget the heuristic therefore dominates the exact solver on the constrained topologies and matches it on the trivial one. Second, the gap widens sharply with the fleet size: on *Triangle* $R=20$ **Safe** returns $\bar{f} = 677$ against **MILP**’s 2425, and on *Full* $R=20$ the gap is 963 versus 4676, both more than three times apart. Third, the topology heuristic alone (**Topo**) already matches the safe-pipeline value on most $R=10$ rows; on *Chain*, *Two rows*, *Hub* and *Full* a single FAS pass shaves a few percent off Topo, and on *Triangle* $R=30$ **Safe** ends below both Topo and FAS in the mean because it picks the per-seed minimum — this is by construction and illustrates the value of the safe-pipeline composition.

6.4.3 Relative gap to the best solution

To compare methods on a common scale we compute, for each instance, the *relative gap* of each method to the best objective achieved by the four methods on that instance. The gaps in Table 6 confirm that, within the 60s budget, the exact solver is the worst performer on every configuration with non-trivial blocking: from 24 % on *Triangle* $R=10$ up to 40 % on *Hub* $R=10$ and 79 % on *Full* $R=10$, and dramatic gaps on the larger instances (282 % on *Triangle* $R=20$, 431 % on *Full* $R=20$), driven by the very weak feasible incumbents Gurobi returns at the time

Table 5: Mean results over 6 seeds per configuration under the default weight profile $(W^M, W^D, W^S) = (0.1, 1, 10)$. Columns per method: mean objective $\bar{f}$, mean makespan $\bar{m}$, mean total delay $\bar{v}^D$, mean movements $\bar{n}$, and mean wall-clock time $\bar{t}$ (s). MILP is unavailable for $R = 30$.

Configuration	MILP					Topo					FAS					Safe				
	$\bar{f}$	$\bar{m}$	$\bar{v}^D$	$\bar{n}$	$\bar{t}$	$\bar{f}$	$\bar{m}$	$\bar{v}^D$	$\bar{n}$	$\bar{t}$	$\bar{f}$	$\bar{m}$	$\bar{v}^D$	$\bar{n}$	$\bar{t}$	$\bar{f}$	$\bar{m}$	$\bar{v}^D$	$\bar{n}$	$\bar{t}$
None ($R=10$, tight)	109.28	66.17	102.67	0.0	69.1	107.22	65.50	100.67	0.0	60.0	107.22	65.50	100.67	0.0	0.0	107.22	65.50	100.67	0.0	0.0
Chain ($R=10$, tight)	220.17	90.00	197.83	1.3	69.4	177.32	85.67	158.75	1.0	60.0	174.12	86.17	155.50	1.0	0.2	174.12	86.17	155.50	1.0	0.0
Hub ($R=10$, tight)	197.60	90.17	188.58	0.0	80.9	142.16	82.42	133.92	0.0	60.0	142.03	82.75	133.75	0.0	0.1	142.03	82.75	133.75	0.0	0.0
Triangle ($R=10$, tight)	145.82	79.08	137.92	0.0	68.3	116.78	69.50	109.83	0.0	60.0	116.78	69.50	109.83	0.0	0.0	116.78	69.50	109.83	0.0	0.0
Two rows ($R=10$, tight)	146.71	77.92	138.92	0.0	81.9	116.15	68.17	109.33	0.0	60.1	113.08	67.50	106.33	0.0	0.4	113.08	67.50	106.33	0.0	0.0
Full ($R=10$, tight)	303.73	119.75	275.08	1.7	68.6	195.76	86.75	180.42	0.7	60.0	185.63	86.33	173.67	0.3	0.4	185.63	86.33	173.67	0.3	0.0
Triangle ($R=5$, tight)	3.55	35.50	0.00	0.0	4.9	3.72	35.50	0.17	0.0	60.1	3.72	35.50	0.17	0.0	0.4	3.72	35.50	0.17	0.0	0.0
Triangle ($R=20$, tight)	2425.10	321.00	2393.00	0.0	194.4	687.12	132.00	673.92	0.0	63.1	677.12	132.00	663.92	0.0	23.4	677.12	132.00	663.92	0.0	0.0
Triangle ($R=30$, tight)	—	—	—	—	—	2140.12	231.25	2107.00	1.0	588.0	2146.20	234.50	2092.75	3.0	60.5	2119.82	234.08	2079.75	1.7	0.0
Triangle ($R=10$, medium)	140.62	94.58	131.17	0.0	70.3	74.38	68.83	67.50	0.0	60.0	74.38	68.83	67.50	0.0	0.1	74.38	68.83	67.50	0.0	0.0
Triangle ($R=10$, loose)	17.50	70.83	10.42	0.0	68.5	15.14	71.42	8.00	0.0	60.0	15.14	71.42	8.00	0.0	0.1	15.14	71.42	8.00	0.0	0.0
Full ($R=20$, tight)	4675.60	531.00	4622.50	0.0	156.2	992.38	167.08	962.33	1.3	62.8	972.98	168.17	922.83	3.3	65.6	963.27	170.25	926.25	2.0	0.0

limit. **Topo** alone stays below 7 % on every $R=10$ tight configuration and below 4 % at $R=20$; **FAS** closes the residual gap to 0 % in every case except *Triangle* $R=30$, where Topo and FAS each win on some seeds and only the Safe pipeline attains 0 % throughout. Two apparent outliers stand out: every method sits at 41.6 % on *Triangle* $R=10$, *loose*, and at 5.2 % on *Triangle* $R=5$ (Topo/FAS/Safe). Both come from seeds whose best objective is close to zero, so even tiny absolute deviations translate to large relative gaps; the absolute differences in Table 5 remain well within a few units.

Table 6: Mean relative gap (%) of each method with respect to the best objective found across the four methods on the same seed, under the default weight profile. The Safe pipeline is the per-seed minimum of Topo and FAS, so its gap is always $\leq$ both. Lower is better.

Configuration	MILP	Topo	FAS	Safe
None ($R=10$, tight)	1.87	0.00	0.00	0.00
Chain ($R=10$, tight)	26.76	1.96	0.00	0.00
Hub ($R=10$, tight)	40.40	0.23	0.13	0.13
Triangle ($R=10$, tight)	24.38	0.00	0.00	0.00
Two rows ($R=10$, tight)	29.36	2.48	0.00	0.00
Full ($R=10$, tight)	78.91	6.81	0.00	0.00
Triangle ($R=5$, tight)	0.00	5.21	5.21	5.21
Triangle ($R=20$, tight)	281.91	1.49	0.00	0.00
Triangle ($R=30$, tight)	—	1.01	1.24	0.00
Triangle ($R=10$, medium)	89.41	0.00	0.00	0.00
Triangle ($R=10$, loose)	63.40	41.57	41.57	41.57
Full ($R=20$, tight)	430.87	3.17	0.98	0.00

6.4.4 Weight-profile sensitivity

Table 7 isolates the effect of the three weight profiles on the structure of the safe-pipeline solutions. Under the default, movement-priority profile, the safe pipeline returns essentially zero blocking movements on every configuration covered by the benchmark — the only non-zero entries are residuals of $\bar{n}=1.0$ on *Chain* $R=10$, $\bar{n}=0.3$ on *Full* $R=10$, $\bar{n}=1.7$ on *Triangle* $R=30$ and $\bar{n}=2.0$ on the dense *Full* $R=20$ — at the cost of a moderate makespan penalty: e.g. $\bar{m}=86.3$

on *Full* $R=10$ versus 62.4 under wC, and $\bar{m}=170.3$ on *Full* $R=20$ versus 119.8. Switching to the delay-priority profile (wB) collapses the mean delay onto a narrow band — $\bar{v}^D \approx 100.7$ on every $R=10$ tight row regardless of topology — but injects 6–45 blocking movements per instance. The makespan-priority profile (wC) flattens $\bar{m}$ to about 62.3–62.4 on every $R=10$ tight row at the price of 3–28 movements and intermediate delay. The three profiles therefore act as effective dials between the objectives, and the default profile is the appropriate choice when the operational priority is to keep aircraft in the assigned slot for the whole servicing window.

Table 7: Effect of the weight profile on the Safe pipeline solution structure. Mean $\bar{m}$, mean total delay $\bar{v}^D$, and mean movements $\bar{n}$ (averaged over 6 seeds) under the three profiles: default (0.1, 1, 10), wB (1, 10, 0.1) (delay-priority), wC (10, 0.1, 1) (makespan-priority).

Configuration	Default			wB (delay)			wC (makespan)		
	$\bar{m}$	$\bar{v}^D$	$\bar{n}$	$\bar{m}$	$\bar{v}^D$	$\bar{n}$	$\bar{m}$	$\bar{v}^D$	$\bar{n}$
None ($R=10$, tight)	65.50	100.67	0.0	64.67	100.67	0.0	62.33	103.33	0.0
Chain ($R=10$, tight)	86.17	155.50	1.0	64.67	100.67	19.0	62.33	110.00	12.0
Hub ($R=10$, tight)	82.75	133.75	0.0	65.50	100.67	16.7	62.33	116.50	10.7
Triangle ($R=10$, tight)	69.50	109.83	0.0	64.67	100.67	11.7	62.33	109.50	7.3
Two rows ($R=10$, tight)	67.50	106.33	0.0	65.50	100.67	6.0	62.33	108.67	3.3
Full ($R=10$, tight)	86.33	173.67	0.3	65.50	100.67	45.3	62.42	123.92	28.0
Triangle ($R=5$, tight)	35.50	0.17	0.0	35.50	0.00	0.3	35.50	4.67	0.0
Triangle ($R=20$, tight)	132.00	663.92	0.0	125.42	621.58	32.0	118.17	648.67	25.0
Triangle ($R=30$, tight)	234.08	2079.75	1.7	217.90	1945.00	61.6	207.10	2002.60	40.4
Triangle ($R=10$, medium)	68.83	67.50	0.0	65.67	54.00	11.0	62.33	65.83	6.7
Triangle ($R=10$, loose)	71.42	8.00	0.0	67.50	0.50	9.0	62.33	11.58	8.7
Full ($R=20$, tight)	170.25	926.25	2.0	125.92	623.83	123.7	119.83	688.17	76.0

7 Conclusions

We have introduced the aircraft hangar positioning problem (AHPP) and provided a formal graph-based model centred on directed blocking arcs, a compact mixed-integer linear programming formulation expressed at the aircraft level (each aircraft as a single uninterrupted maintenance block) that encodes entry and exit movements through a six-binary indicator structure, and a suite of complementary heuristic methods built on top of the same data model: a topology-aware GRASP that scores positions by their blocking load, a fixed-assignment scheduler (FAS) that re-optimises timing given a position assignment, and a safe pipeline composition that returns the best of the components at no extra cost.

The computational study on the 120-instance benchmark gives a clear picture of when each method dominates. Within the 60-second wall-clock budget, the exact MILP only proves optimality on the small $R=5$ configuration; for every $R=10$ configuration the Gurobi gap remains above 90 %, including the unconstrained *None* topology, which points to bound weakness rather than blocking interactions as the dominant source of difficulty. At $R=20$ the solver exhausts the budget with a gap close to 100 %, and at $R=30$ it aborts before returning any feasible incumbent, so the heuristic pipeline becomes the only available source of solutions for the largest configurations. On the $R=10$ configurations with non-trivial blocking the heuristic pipeline cuts the best feasible MILP objective by 5–25 %, and at $R=20$ the gap widens by an order of

magnitude (e.g. $\bar{f} = 635$ versus 2425 on *Triangle*, and 881 versus 4676 on *Full*). The topology heuristic alone already matches the safe-pipeline objective on every $R=10$ row, so the additional FAS pass acts mostly as an insurance policy; its value becomes apparent at the boundary of tractability, where FAS occasionally times out — on *Triangle* $R=30$, for instance, FAS lags Topo by 2.2 % and the safe pipeline correctly falls back to the Topo solution. The three weight profiles act as effective dials between the objectives: the default movement-priority profile returns *zero* blocking movements on every configuration covered by the benchmark at the cost of a moderate makespan penalty; the delay-priority profile flattens mean delay onto a narrow band on the $R=10$ rows while injecting tens of movements; and the makespan-priority profile yields the lowest $\bar{m}$ throughout, again at the cost of intermediate values of delay and movements.

These results outline a natural agenda for future work. A tighter MILP formulation that exploits the capacity-one structure of the positions and tightens the big- M relaxation should close part of the bound gap that currently bottlenecks the exact solver from $R=10$ onwards. An adaptive large neighbourhood search built on top of FAS, with destroy and repair operators selected from observed per-instance-type performance, is a promising avenue for the $R \geq 20$ regime where FAS already starts to feel the time limit. A multi-shift extension that allows aircraft arrivals to span several working days, and a validation on production data from an airline maintenance partner, would complete the bridge from the controlled benchmark presented here to operational deployment.

References

- Ehsan Asadi, Michael Schultz, and Hartmut Fricke. Optimal schedule recovery for the aircraft gate assignment with constrained resources. *Computers & Industrial Engineering*, 162:107682, 2021. doi: 10.1016/j.cie.2021.107682.
- Gastón Bayá, Eduardo Canale, Sergio Nesmachnow, Franco Robledo, and Pablo Sartor. Production optimization in a grain facility through mixed-integer linear programming. *Applied Sciences*, 12(16):8212, 2022. doi: 10.3390/app12168212.
- Jacek Blazewicz, Jan Karel Lenstra, and Alexander H. G. Rinnooy Kan. Scheduling subject to resource constraints: classification and complexity. *Discrete Applied Mathematics*, 5(1):11–24, 1983. doi: 10.1016/0166-218X(83)90012-4.
- Michael L. Bynum, Gabriel A. Hackebeil, William E. Hart, Carl D. Laird, Bethany L. Nicholson, John D. Sirola, Jean-Paul Watson, and David L. Woodruff. *Pyomo — Optimization Modeling in Python*. Springer, 3rd edition, 2021. doi: 10.1007/978-3-030-68928-5.
- Manoj Dahanayaka, Dennis Prak, and Martijn Mes. From gate to runway: A systematic review of airport ground operations optimization. *Journal of Air Transport Management*, 2026. doi: 10.1016/j.jairtraman.2026.103013.
- Jan Evler, Ehsan Asadi, Henning Preis, and Hartmut Fricke. Airline ground operations: Schedule recovery optimization approach with constrained resources. *Transportation Research Part C: Emerging Technologies*, 128:103129, 2021a. doi: 10.1016/j.trc.2021.103129.

- Jan Evler, Ehsan Asadi, Henning Preis, and Hartmut Fricke. Airline ground operations: Optimal schedule recovery with uncertain arrival times. *Journal of Air Transport Management*, 92: 102021, 2021b. doi: 10.1016/j.jairtraman.2021.102021.
- Shayan Farhang Pazhooh and Hossein Shams Shemirani. An efficient continuous-time MILP for integrated aircraft hangar scheduling and layout. arXiv preprint arXiv:2508.02640, 2025.
- Thomas A. Feo and Mauricio G. C. Resende. Greedy randomized adaptive search procedures. *Journal of Global Optimization*, 6(2):109–133, 1995. doi: 10.1007/BF01096763.
- Daniele Ferone, Aljoscha Gruler, Paola Festa, and Angel A. Juan. Enhancing and extending the classical GRASP framework with biased randomisation and simulation. *Journal of the Operational Research Society*, 70(8):1362–1375, 2018. doi: 10.1080/01605682.2018.1494527.
- Paola Festa and Mauricio G. C. Resende. GRASP: An annotated bibliography. In *Essays and Surveys in Metaheuristics*, pages 325–367. Springer, 2002. doi: 10.1007/978-1-4615-1507-4_15.
- Paola Festa and Mauricio G. C. Resende. An annotated bibliography of GRASP — part II: Applications. *International Transactions in Operational Research*, 16(2):131–172, 2009. doi: 10.1111/j.1475-3995.2009.00664.x.
- Julien Guépet, Rodrigo Acuna-Agost, Olivier Briant, and Jean-Philippe Gayon. Exact and heuristic approaches to the airport stand allocation problem. *European Journal of Operational Research*, 246(2):597–608, 2015. doi: 10.1016/j.ejor.2015.04.040.
- Gurobi Optimization, LLC. Gurobi optimizer reference manual. <https://www.gurobi.com>, 2023.
- Leslie A. Hall and David B. Shmoys. Jackson’s rule for single-machine scheduling: making a good heuristic better. *Mathematics of Operations Research*, 17(1):22–35, 1996.
- Minkwan Lee, Sooyung Yu, Kibeom Kwon, Minkyu Lee, Junseok Lee, and Hyochoon Kim. Mixed-integer linear programming model for scheduling missions and communications of multiple satellites. *Aerospace*, 11(1):83, 2024. doi: 10.3390/aerospace11010083.
- Jan Karel Lenstra, Alexander H. G. Rinnooy Kan, and Peter Brucker. Complexity of machine scheduling problems. *Annals of Discrete Mathematics*, 1:343–362, 1977. doi: 10.1016/S0167-5060(08)70743-X.
- Yuesheng Mao, Qiuhua Tang, Zikai Li, and Liping Zhang. Mixed-integer linear programming method for multi-degree and multi-hoist cyclic scheduling with time windows. *Engineering Optimization*, 50(11):1978–1995, 2018. doi: 10.1080/0305215X.2017.1418865.
- Michael L. Pinedo. *Scheduling: Theory, Algorithms, and Systems*. Springer, 5th edition, 2016. doi: 10.1007/978-3-319-26580-3.
- Yichen Qin, Felix T. S. Chan, S. H. Chung, and Ting Qu. Development of MILP model for integrated aircraft maintenance scheduling and multi-period parking layout planning problems. In *Proceedings of the 12th IEEE Conference on Industrial Electronics and Applications (ICIEA)*, 2017. doi: 10.1109/IEA.2017.7939206.

- Yichen Qin, Felix T. S. Chan, S. H. Chung, Ting Qu, and Ben Niu. Aircraft parking stand allocation problem with safety consideration for independent hangar maintenance service providers. *Computers & Operations Research*, 91:225–236, 2018a. doi: 10.1016/j.cor.2017.10.001.
- Yichen Qin, Zhenzhen Wang, Felix T. S. Chan, S. H. Chung, and Ting Qu. A family of heuristic-based inequalities for maximizing overall safety margins in aircraft parking stands arrangement problems. *Mathematical Problems in Engineering*, 2018:3525384, 2018b. doi: 10.1155/2018/3525384.
- Yichen Qin, Zhenzhen Wang, Felix T. S. Chan, S. H. Chung, and Ting Qu. A mathematical model and algorithms for the aircraft hangar maintenance scheduling problem. *Applied Mathematical Modelling*, 67:491–509, 2019. doi: 10.1016/j.apm.2018.11.008.
- Yichen Qin, Jianxia Zhang, Felix T. S. Chan, S. H. Chung, Ben Niu, and Ting Qu. A two-stage optimization approach for aircraft hangar maintenance planning and staff assignment problems under MRO outsourcing mode. *Computers & Industrial Engineering*, 146:106607, 2020. doi: 10.1016/j.cie.2020.106607.
- Syed Shaukat, Mathias Katscher, Cheng-Lung Wu, Felipe Delgado, and Homero Larrain. Aircraft line maintenance scheduling and optimisation. *Journal of Air Transport Management*, 89:101914, 2020. doi: 10.1016/j.jairtraman.2020.101914.
- David Torres Sanchez, Burak Boyacı, and Konstantinos G. Zografos. An optimisation framework for airline fleet maintenance scheduling with tail assignment considerations. *Transportation Research Part B: Methodological*, 133:142–164, 2020. doi: 10.1016/j.trb.2019.12.008.
- Haoqing Wang, Fulgencia Villa, Eva Vallada, and Rubén Ruiz. Solving the yard crane scheduling problem with dynamic assignment of input/output points. *Computers & Operations Research*, 173:106853, 2024. doi: 10.1016/j.cor.2024.106853.
- Marvin Witteman, Qichen Deng, and Bruno F. Santos. A bin packing approach to solve the aircraft maintenance task allocation problem. *European Journal of Operational Research*, 294(1):365–376, 2021. doi: 10.1016/j.ejor.2021.01.027.
- Min Yao, Minghua Hu, Jianan Yin, Junfeng Su, and Mengfan Yin. A two-stage optimization model for airport stand allocation and ground support vehicle scheduling. *Applied Sciences*, 14(23):11407, 2024. doi: 10.3390/app142311407.